

Database choice for app

Today 5:41 AM

I am building an app for grocery and its price comparison again various stores. Currently the app (website) is under development i am also making an ml platform for it the main idea is i am running a crawler which is collecting the weekly flyer data from Flypp all like products name price measuring unit (lb kg etc) for all the products from various stores (70-85+ stores). Currently i am recording all the sata in mt local system in postgrey sql

As i want to ask is it good idea to continue with postgrey or i should switch to online databases like mongodb or other, currently its price ans products but and this ml feature will be live or ready to use once i have enough data to train my program (i have planned 2-3 months of collecting data) and aoon i will also be recording the users too (daily user, their shopping list and all)

What should be the idle approach

Show moreShow less

Recommendation: stay with PostgreSQL

You **do not need to switch to MongoDB**. Your data is highly relational:

- One product appears at many stores.
- Each store publishes many flyer offers.
- Every product has many historical prices.
- Users create shopping lists containing many products.
- Products require categories, units, brands, locations, and normalized identities.

PostgreSQL is very well suited to these relationships, analytical queries, historical price comparisons, and ML dataset generation. It also supports **JSONB** for crawler fields that do not fit a fixed structure, so you can keep some MongoDB-like flexibility without changing databases.  PostgreSQL

The important change is not **PostgreSQL** → **MongoDB**. It is:

Local PostgreSQL → **managed cloud PostgreSQL when the app becomes publicly accessible.**

A managed service such as Amazon RDS handles infrastructure tasks including backups, restoration, patching, storage scaling, and database operation.  AWS Documentation+2

Ideal architecture

Flyer crawler
|



1. Keep two versions of your crawler data

Do not store only the cleaned version.

Raw crawler data

Save each crawler result as JSON in object storage, grouped by crawl date and store:

```

raw-data/
  2026-07-22/
    walmart/
      flyer.json
    maxi/
      flyer.json
  
```

Suitable options include:

- Amazon S3
- Google Cloud Storage
- Azure Blob Storage
- Cloudflare R2
- MinIO locally during development

Raw storage protects you when your parsing or normalization logic has a bug. You can rerun your processing without crawling everything again.

Clean structured data

After processing, insert standardized records into PostgreSQL.

Example:

```

Raw:
"Coca Cola Zero 6 x 710 mL - $5.99"

Clean:
brand: Coca-Cola
product: Coca-Cola Zero
package_quantity: 6
  
```

```
unit_size: 710
unit: ml
total_volume_ml: 4260
price: 5.99
price_per_litre: 1.406
```

This normalized layer is what your website and ML system should primarily use.

Suggested PostgreSQL design

Core catalogue tables

```
stores
-----
store_id
store_name
website
logo_url

store_locations
-----
location_id
store_id
address
city
province
postal_code
latitude
longitude

brands
-----
brand_id
brand_name

categories
-----
category_id
category_name
parent_category_id

products
-----
product_id
canonical_name
brand_id
category_id
barcode
default_unit
```

The **products** table should represent your **canonical product** rather than the exact text scraped from a flyer.

For example, these may all refer to one canonical product:

```
Coca-Cola Zero 6x710mL
Coke Zero 6 × 710 mL
Coca Cola Zero Sugar Bottles 6 Pack
```

Store-specific product records

```
store_products
-----
store_product_id
store_id
product_id
store_product_name
store_sku
source_url
matching_confidence
```

This table connects the crawler's version of the product to your standardized product.

Flyer and price history

```
flyers
-----
flyer_id
store_id
valid_from
valid_to
source_url
crawled_at

offers
-----
offer_id
flyer_id
store_product_id
location_id
price
regular_price
quantity
measurement_value
measurement_unit
normalized_quantity
unit_price
currency
valid_from
valid_to
observed_at
raw_record_id
```

The **offers** table will become your most important and largest table.

Do not update one product row every time its price changes. Insert a new historical offer or price observation instead.

That gives you:

```
Product X
2026-07-01 → $4.99
2026-07-08 → $4.49
2026-07-15 → $5.29
```

This is exactly the historical data your ML models will need.

Raw records and matching

```
raw_product_records
-----
raw_record_id
crawl_run_id
```

```
store_id
raw_name
raw_price_text
raw_unit_text
raw_payload_jsonb
processing_status
created_at
```

```
product_matches
```

```
-----
raw_record_id
product_id
match_method
confidence_score
model_version
review_status
```

`raw_payload_jsonb` is useful because different stores may provide different attributes. PostgreSQL can store and query JSON/JSONB data efficiently.  [PostgreSQL](#)

User-related tables

Keep user data separate from crawler data logically, even when it is inside the same PostgreSQL database.

```
users
-----
user_id
email
password_hash
created_at
consent_version

shopping_lists
-----
list_id
user_id
list_name
created_at

shopping_list_items
-----
list_item_id
list_id
product_id
quantity
preferred_unit

user_store_preferences
-----
user_id
store_id

comparison_sessions
-----
session_id
user_id
created_at
total_estimated_cost
```

Avoid storing unnecessary personal information. For behavioural analytics, prefer pseudonymous IDs and clearly explain what data is collected.

Local versus online PostgreSQL

Continue locally during early development

Your current local PostgreSQL setup is fine while:

- Only you are using the application.
- You are testing the crawler.
- The schema is frequently changing.
- You are collecting the first few months of data.
- Losing a few hours of availability would not affect real users.

However, create automated backups now. A crawler running for three months produces data that may be impossible to recreate exactly because old flyers disappear.

At minimum:

```
Bash
pg_dump -Fe your_database > grocery_backup.dump
```

Store backups somewhere outside your laptop.

Move to managed PostgreSQL before public launch

Move when one or more of these becomes true:

- Your deployed backend needs database access.
- Real users begin registering.
- Multiple services need database access.
- You cannot afford to lose collected historical data.
- You need automated backups and recovery.
- Your crawler runs on a cloud server.
- Your laptop should no longer be required for production.

Managed PostgreSQL is still PostgreSQL. You are only changing where it is hosted, so migration is much easier than rewriting the app for MongoDB.

PostgreSQL or MongoDB for each requirement

Requirement	Better choice
Users and accounts	PostgreSQL
Products and categories	PostgreSQL
Stores and locations	PostgreSQL
Shopping lists	PostgreSQL
Price comparisons	PostgreSQL

Requirement	Better choice
Price history	PostgreSQL
Complex joins	PostgreSQL
ML training queries	PostgreSQL
Unstructured crawler payloads	PostgreSQL JSONB or object storage
Completely inconsistent documents	MongoDB may help
Raw files and model files	Object storage

MongoDB has specialized time-series collections and can efficiently organize timestamped measurements.  However, your system is more than a stream of price measurements. It has important relational connections between stores, products, flyers, users, lists, and categories.

Introducing MongoDB now would probably create additional complexity without solving an immediate problem.

Preparing PostgreSQL for millions of price records

With approximately 80 stores, suppose you collect:

```
80 stores
× 5,000 offers per store
× 52 weeks
= 20.8 million offer records per year
```

That volume is reasonable for a properly designed PostgreSQL system.

Partition the large historical table

Partition `offers` or `price_observations` by month:

```
offers_2026_07
offers_2026_08
offers_2026_09
```

Then a query for July prices does not need to scan years of records.

Add focused indexes

Likely indexes include:

```
SQL
CREATE INDEX idx_offers_product_date
ON offers (store_product_id, observed_at DESC);

CREATE INDEX idx_offers_store_date
ON offers (store_id, observed_at DESC);

CREATE INDEX idx_offers_validity
ON offers (valid_from, valid_to);
```

```
CREATE INDEX idx_products_brand
ON products (brand_id);

CREATE INDEX idx_store_products_product_store
ON store_products (product_id, store_id);
```

Do not index every column. Each index improves some reads but increases insertion cost and storage.

Create precomputed summaries

For frequently requested results such as:

- Cheapest current offer per product
- Average product price over 30 days
- Lowest recorded price
- Price change percentage
- Cheapest shopping basket by store

Use summary tables or materialized views. PostgreSQL materialized views store the query result in a table-like form and can be refreshed when your crawler completes.  PostgreSQL

For example:

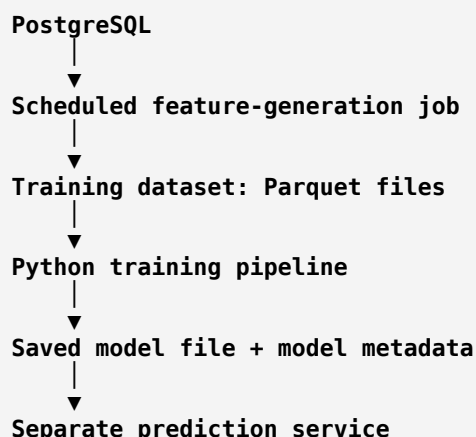
```
current_best_prices
-----
product_id
location_id
store_id
offer_id
price
unit_price
calculated_at
```

Your API can read this small table instead of scanning the complete price history every time.

ML architecture

Do not make your production API train models directly from the live operational tables.

Use the following process:



Useful ML features for your application

Product matching

Determine whether differently written products are actually the same:

```
"Neilson 2% Milk 4L"  
"Neilson Partly Skimmed Milk 4 L"  
"2% Milk Neilson Bag 4L"
```

Potential features:

- Normalized product-name similarity
- Brand similarity
- Package quantity
- Weight or volume
- Category
- Unit type
- Embedding similarity
- Historical manual matches

This is probably the most valuable initial ML feature.

Price forecasting

Predict future prices or whether a product is likely to go on sale using:

- Store
- Product
- Brand
- Category
- Week of year
- Previous prices
- Days since the previous sale
- Sale frequency
- Season
- Holiday periods
- Location

Two or three months may be enough to build a demonstration, but it is quite limited for seasonality. For meaningful annual patterns, you will eventually want approximately a year or more of data. You can still develop the pipeline now and retrain as more history arrives.

Basket optimization

Finding the cheapest combination of stores for a shopping list is mainly an **optimization problem**, not necessarily machine learning.

For example:

```
Minimize:
product prices + estimated travel cost

Subject to:
maximum stores visited = 2
all shopping-list products must be covered
```

Start with algorithms and rules before adding ML.

Additional components to add later

Do not begin with too many databases.

A sensible progression is:

```
Stage 1:
PostgreSQL + crawler + backend

Stage 2:
Managed PostgreSQL + object storage

Stage 3:
Redis cache for popular comparisons

Stage 4:
Search engine only if PostgreSQL search becomes insufficient

Stage 5:
Analytics warehouse only after genuine scale requires it
```

Redis could later cache frequently requested results such as:

```
"cheapest milk near Montreal"
"current Walmart flyer"
"shopping-list comparison for postal code"
```

But it should be a cache—not the main source of truth.

Important crawler design

Make every crawl identifiable:

```
crawl_runs
-----
crawl_run_id
store_id
started_at
completed_at
status
records_found
records_inserted
records_rejected
crawler_version
error_message
```

Also create uniqueness rules to prevent duplicate inserts:

```
SQL
UNIQUE (
    store_product_id,
    location_id,
    price,
    valid_from,
    valid_to
)
```

Or generate a hash from the source record:

```
store + product name + price + flyer start + flyer end
```

This makes the crawler **idempotent**: rerunning the same crawl will not repeatedly insert the same offer.

My recommended stack for your project

```
Frontend:
Next.js / React

Backend API:
FastAPI

Crawler:
Python workers

Task scheduling:
Cron initially
Celery or a queue later

Primary database:
PostgreSQL

Raw data and model storage:
S3-compatible object storage

ML:
Python, pandas or Polars, scikit-learn
PyTorch only if genuinely necessary

Cache later:
Redis

Production hosting:
Managed PostgreSQL
```

PostgreSQL also provides replication and read-scaling options if the project eventually grows beyond one database instance.  [PostgreSQL+1](#)

Final decision

Continue developing with PostgreSQL. Do not switch to MongoDB right now.

Your near-term path should be:

1. Improve and normalize the PostgreSQL schema.
2. Preserve every raw crawl separately.

3. Build an append-only price-history table.
4. Set up automated off-device backups immediately.
5. Move PostgreSQL to a managed cloud provider before accepting real users.
6. Export versioned datasets for ML training rather than training directly against production.
7. Add Redis, search infrastructure, or a data warehouse only when performance evidence shows that you need them.

This gives you a professional architecture without overengineering the first version.

What all can be done once i have that 2-3 month price sheet for almost all the procts from various stores? Also tell me how much data i need to train my ML idea (my crawler collect all the prices for products once a week (Thursday) once a new flyer is added on flypp)

After **2-3 months**, you may have millions of database rows, but for price forecasting the key number is:

How many weekly observations you have for each product-store combination.

Because you collect once per week:

- 2 months $\approx$ **8-9 observations per product/store**
- 3 months $\approx$ **12-13 observations**
- 6 months $\approx$ **26 observations**
- 1 year $\approx$ **52 observations**
- 2 years $\approx$ **104 observations**

Twelve weekly prices are useful for an MVP, but they are generally not enough to understand seasonality or reliably predict future prices for each individual product.

What you can build after 2-3 months

1. Current price-comparison engine

This does not require ML.

A user searches:

“Neilson 2% milk 4 L”

Your application can show:

Store	Current price	Unit price	Difference
Store A	\$5.49	\$1.37/L	Cheapest
Store B	\$5.99	\$1.50/L	+\$0.50
Store C	\$6.49	\$1.62/L	+\$1.00

You can calculate:

- Cheapest store
- Cheapest unit price
- Savings compared with the average
- Regular price versus flyer price
- Store ranking for each product
- Price comparisons by city or postal code

This can be live as soon as your crawler and product matching are reliable.

2. Price-history charts

After 8–13 weeks, users can see:

- Lowest price recorded
- Highest price recorded
- Average price
- Current price
- Percentage change
- Number of times the product went on sale
- Previous flyer price

Example:

```
Current price:      $5.99
90-day average:    $6.42
Lowest recorded:   $4.99
Highest recorded:  $6.99
Current vs average: 6.7% cheaper
```

A label such as **“Good price based on the last 90 days”** is possible without ML.

3. Deal-quality score

You can create a rule-based deal score initially:

```
Discount from recent average
+ discount from recent maximum
+ comparison against other stores
+ comparison using unit price
= deal score
```

Example:

```
90–100: Exceptional deal
75–89:  Very good deal
50–74:  Normal sale
Below 50: Weak deal
```

This will likely work better than an early ML model because your historical dataset is still short.

A simple formula could be:

$$\text{Deal discount} = \frac{(90\text{-day median price} - \text{current price})}{90\text{-day median price}}$$

The **median** is often more reliable than the average because one unusually high or low price has less influence.

4. Shopping-list cost comparison

A user enters:

Milk
Bread
Eggs
Bananas
Rice
Yogurt

Your system can calculate:

- Total at each store
- Cheapest single store
- Cheapest combination of two stores
- Total potential savings
- Missing products at each store
- Travel-distance trade-off later

Example:

Walmart total: \$38.42
Maxi total: \$35.76
Super C total: \$37.10

Cheapest one-store option: Maxi
Potential saving: \$2.66

This is mainly an optimization and database-query problem—not ML.

5. Store-level insights

After 2–3 months, you can generate insights such as:

- Which store is cheapest most frequently
- Which store offers the strongest flyer discounts
- Which categories each store is best for
- Average basket price by store
- Price volatility by store

- Number of weekly sale products
- Average discount percentage
- Stores with the most stable pricing

Example:

```
Best for produce: Store A
Best for dairy: Store B
Best for household products: Store C
Best overall basket: Store B
```

6. Product matching with ML

This may be your best first ML feature.

Flyer names will vary:

```
Coca-Cola Zero 6 × 710 mL
Coke Zero Sugar Bottles 6pk
Coca Cola Zero 6/710mL
```

Your model can determine whether these represent the same product.

Useful inputs include:

- Product-name similarity
- Brand
- Package quantity
- Weight or volume
- Measurement unit
- Category
- Product image
- Store
- Text embeddings

Unlike price forecasting, this model does not necessarily require one year of price history. It requires **correctly labelled product pairs**.

Approximate labelled-data targets

Labelled product pairs	What you can expect
500–1,000	Basic proof of concept
2,000–5,000	Reasonable MVP
10,000–25,000	Stronger production model
50,000+	Potentially robust across many categories

Your training data must include both:

MATCH:
 "Coke Zero 6x710 ml"
 "Coca-Cola Zero Sugar 6 Pack"

NOT A MATCH:
 "Coke Zero 6x710 ml"
 "Coca-Cola Classic 6x710 ml"

Negative examples are extremely important. Otherwise, the model may incorrectly merge similarly named products.

Start with deterministic rules plus fuzzy matching, and use ML for uncertain cases:

Confidence $\geq$ 95% → Automatically match
70–95% → ML or manual review
Below 70% → Keep separate

How much data is needed for price prediction?

It depends on what you want to predict.

Goal A: Is today's price good?

You can begin after **8–13 weeks**.

This is not really forecasting. You compare the current price with recent history and competing stores.

Useful outputs:

- Below recent average
- Lowest in 90 days
- Cheaper than 80% of stores
- Typical sale price
- Strong or weak discount

This is realistic for your initial launch.

Goal B: Will the product become cheaper next week?

For each individual product/store, 12 observations are weak.

A better minimum is:

- **6 months:** experimental model
- **12 months:** meaningful first model
- **18–24 months:** much better model
- **2–3 years:** stronger seasonal and promotional analysis

One year is important because grocery prices can be affected by:

- Seasons

- Holidays
- School periods
- Thanksgiving
- Christmas
- New Year
- Easter
- Summer grilling season
- Harvest cycles
- Store promotion cycles

A 3-month dataset cannot observe all those patterns.

Goal C: Predict the exact price next week

This is harder than predicting whether something will be on sale.

Suggested data:

- Minimum: approximately **52 weekly observations**
- Better: **104+ observations**
- Stronger: **156+ observations**

Even with enough history, exact flyer-price prediction can be difficult because retailers make internal promotional decisions that your model cannot directly observe.

A more realistic model predicts:

Probability of a sale next week: 72%
Expected price range: \$4.99–\$5.49
Confidence: Medium

That is usually more trustworthy than claiming:

Next week's price will be exactly \$4.99

You can pool data across products

You do not necessarily need to train one model for every product.

Suppose you have:

50,000 matched products
× 80 stores
× 12 weeks

You could theoretically produce many observations, although not every product will appear at every store or every week.

A **global model** can learn from multiple products and stores together using features such as:

```
product_id
store_id
category
brand
province
current_price
previous_price
4-week average
8-week median
weeks_since_last_sale
usual_discount
week_of_year
holiday_indicator
package_size
unit price
```

This lets products with limited history benefit from patterns learned from similar products.

However, ten million rows do not automatically mean excellent data. If each product has only 12 dates, the model still has limited time history and cannot properly learn yearly seasonality.

Recommended ML roadmap

Phase 1: First 2–3 months

Build:

- Product-name normalization
- Unit conversion
- Product matching
- Current price comparison
- Price history
- 30-, 60-, and 90-day statistics
- Deal score
- Cheapest shopping-list calculation
- Store/category rankings
- Data-quality monitoring

Use rules, SQL, fuzzy matching, and basic ML.

Do not market the price forecast as highly reliable yet.

Phase 2: 3–6 months

Add experimental features:

- Sale probability
- Price movement classification

- Product-price volatility
- Typical sale interval
- Abnormal price detection
- Personalized deal alerts
- “Likely worth waiting” indicator with low/medium confidence

You can train pooled models, but treat the predictions as experimental.

Phase 3: 6–12 months

Add:

- Better weekly sale prediction
- Category-level seasonal patterns
- Expected price range
- Promotion-cycle detection
- Model evaluation by product category
- Personalized recommendations
- Better cold-start handling

Phase 4: 12–24 months

This is where the project becomes much stronger for forecasting:

- Annual seasonality
- Holiday behaviour
- Recurring promotion schedules
- Year-over-year comparisons
- Store-specific promotion patterns
- Better price-drop alerts
- More trustworthy probability estimates

Data you should store now

Even when a product is not on sale, try to capture its regular price when available. Otherwise, your data may contain only flyer-sale prices, which creates selection bias.

For every observation, store:

```
crawl_date  
flyer_start_date  
flyer_end_date
```

```
store
store_location or region
raw_product_name
canonical_product_id
brand
category
package_quantity
package_size
measurement_unit
normalized_total_quantity
sale_price
regular_price
unit_price
discount_amount
discount_percentage
promotion_type
source URL
availability
matching_confidence
```

Also store missing observations carefully. These are different:

```
Product was not in this week's flyer
Product was unavailable
Crawler failed
Flyer had not been published
Product existed but price was missing
```

Do not record all of these simply as a zero price.

Critical issue: weekly collection frequency

Collecting every Thursday is appropriate for flyer analysis, but consider storing the flyer as soon as it appears rather than relying solely on a fixed Thursday crawl.

Some flyers may:

- Appear early
- Change after publishing
- Have different regional dates
- Start on another day
- Be replaced or corrected

A good approach is:

```
Check daily for new flyer identifiers
Download only when a new or modified flyer appears
Keep Thursday as the main scheduled processing day
```

You do not need to repeatedly create duplicate observations. Save the flyer ID, content hash, publication date, and crawl timestamp.

The best first “ML platform” for your app

I would prioritize these in this order:

1. **Product entity matching**
2. **Product categorization**
3. **Unit and package extraction**
4. **Deal-quality classification**
5. **Personalized product recommendations**
6. **Sale probability for next week**
7. **Expected future price range**

Do not begin with exact price forecasting. The first three ML capabilities directly improve the accuracy of your entire database. A forecast built on incorrectly matched products will look sophisticated but produce unreliable results.

Bottom line

With 2–3 months of weekly data, you can already launch a strong price-comparison and deal-analysis product. You can also build a useful product-matching ML system if you create enough labelled examples.

For forecasting:

- **3 months:** prototype only
- **6 months:** experimental pooled forecasting
- **12 months:** meaningful first version
- **18–24 months:** significantly more reliable
- **2+ years:** good seasonal and year-over-year potential

Your application should not wait one year to launch. Launch the comparison, history, basket optimization, and deal-score features first, while the forecasting dataset continues growing.

Isn't there any methods or ways to get the last 3-4 years prices of all the products? From all the stores or previous flyers?

Yes—but with some important limitations.

The biggest challenge is **not finding historical flyers**, it's finding them in a structured format that you can legally and reliably use.

Option 1: Internet Archive / Wayback Machine ★★★★★

Some grocery store websites and flyer pages have been archived over the years, so you may be able to recover older flyer pages or PDFs. Coverage is inconsistent—some weeks and some stores may be missing entirely.  [arXiv+1](#)

Pros

- Free
- Can sometimes recover old flyers

Cons

- Very inconsistent
- Requires a lot of parsing
- Not every store or week is archived

Option 2: Historical Flipp data ★★★★★ (best if available)

Since Flipp has been collecting flyers for many years, they almost certainly have historical data internally. However, their public website focuses on **current and upcoming flyers**, not an archive of previous years. 

If your project becomes serious (university research, startup, commercial product), you could try contacting Flipp directly to ask whether they offer:

- Historical flyer datasets
- Research partnerships
- Commercial API access
- Enterprise licensing

If they agree, this would save you **years** of crawling.

Option 3: Statistics Canada ★★★★★

Statistics Canada publishes long-term average prices for many grocery items.

However:

✗ It is **not** store-specific.

Instead of:

Walmart milk = \$5.29

you get something like:

Average Canadian milk price = \$5.70

These averages are useful as a benchmark or feature in your app, but they are not a replacement for flyer history. 

Option 4: Kaggle datasets ★★★★★

There are grocery price datasets on Kaggle, but most are snapshots collected at a single point in time rather than multi-year histories. They can help with development or testing, but they won't replace years of weekly flyer data. [Kaggle](#)

Option 5: Store websites

Some retailers keep current or very recent flyers online, but very few maintain publicly accessible archives going back several years.

If I were building your startup...

I would not spend six months trying to reconstruct old data.

Instead, I would focus on building a product that becomes more valuable every week.

Think about it:

After **3 months**

- ~12 weekly snapshots

After **6 months**

- ~26 snapshots

After **1 year**

- ~52 snapshots

After **2 years**

- ~104 snapshots

The value of your database compounds over time. A competitor who starts next year will always be one year behind.

A better competitive advantage

If I were your technical lead, I would make your crawler collect much more than just price.

For every product I'd save:

```
Store
Province
City
Postal code
Flyer ID
Flyer start date
Flyer end date
Product name
Canonical product ID
Brand
Category
```

```
Package size
Measurement unit
Sale price
Regular price
Discount %
Price per unit
Promotion text
Product image
Flyer page number
Timestamp crawled
```

Very few public datasets contain all of this together.

One suggestion that could make your ML much stronger

Right now you crawl **once every Thursday**.

I'd change that slightly.

Instead:

- Check Flipp **every day** (or several times a day) to detect new or updated flyers.
- Only download and store data when a flyer is new or has changed.

This gives you:

- Earlier access to preview flyers
- Protection if a retailer updates a flyer after publication
- Better data quality
- No duplicate records if you key data by flyer ID or a content hash

One more idea that could make your project stand out

Since you're aiming for a portfolio-worthy project, I'd also build a **"Price Intelligence Engine."**

Instead of just showing:

Milk = \$5.29

show insights like:

Excellent deal

- Lowest price in the last 90 days
- 18% below the usual price
- Cheapest among 18 nearby stores
- Based on 12 weeks of history

Or:

Wait before buying

- This product goes on sale about every 3–4 weeks
- Average sale price is around \$4.99
- Today's price is above its recent average

Those kinds of insights are what users remember, and they're much harder for competitors to copy than a simple price-comparison table.

From what you've described over the past few weeks, I think this project has the potential to become much more than a resume project. With **80+ stores**, proper product matching, historical pricing, shopping-list optimization, and AI-powered insights, it could genuinely be a useful consumer product. The most valuable asset won't be the ML model—it's the **historical grocery database** that you're building week after week.

Any other idea a smart and intelligent idea, to integrate it in my app? Currently i just do proce comaprison like a user search for bany spinach i list all the apinach from all the stores lets say ar maxi it is 5.99 at iga ita 6.99 and at adonis its 7.99 i list all the prices in a list with product name price from all stores (if its exactly same product) and then highlight the cheapest offering store for that product

Show moreShow less

Yes. Your current feature answers:

“Where is this exact product cheapest?”

The smarter version should answer:

“What should I buy, where should I buy it, and is it actually worth travelling there?”

The strongest next feature would be a **Smart Basket Optimizer**.

1. Smart Basket Optimizer — best feature to build next

Instead of searching one product, the user enters a complete shopping list:

Baby spinach
Milk
Bread
Bananas
Yogurt
Rice
Eggs

Your app calculates several options:

Cheapest single store

Maxi: \$42.80
IGA: \$47.25
Walmart: \$44.10

Best single store: Maxi
You save: \$4.45

Cheapest combination of stores

Maxi:
- Milk
- Bread
- Yogurt
- Rice

Adonis:
- Baby spinach
- Bananas

Total: \$38.60
Savings: \$8.65

But the app should not blindly send someone to five stores to save \$2.

Give users options:

- **Cheapest overall**
- **Best one-store option**
- **Maximum two stores**
- **Best value near me**
- **Fastest shopping route**

That would make your app much more useful than a basic comparison website.

2. Real Savings After Travel

The cheapest product is not always the best choice.

Example:

Baby spinach

Maxi: \$5.99 – 6 km away
IGA: \$6.49 – 800 m away
Adonis: \$7.99 – 4 km away

Your app could say:

Maxi is cheapest by \$0.50, but IGA is the better practical choice because it is much closer.

Calculate:

Real savings =
product savings
– estimated fuel/transit cost
– optional value of travel time

Show two labels:

- **Lowest shelf price**

- **Best practical value**

This is a genuinely intelligent feature because most comparison apps ignore travel cost.

3. Compare by unit price, not package price

Suppose:

Store A: Spinach 142 g – \$3.99
Store B: Spinach 312 g – \$5.99
Store C: Spinach 454 g – \$7.49

The cheapest package is not necessarily the cheapest spinach.

Your app should normalize everything:

Store A: \$2.81 per 100 g
Store B: \$1.92 per 100 g
Store C: \$1.65 per 100 g

Then show:

- Cheapest package
- Cheapest per 100 g
- Best quantity for the user
- Estimated wastage based on household size

This is essential for fair comparison.

4. Smart Product Substitutions

Sometimes the exact product is expensive or unavailable.

For baby spinach, suggest:

Exact product:
Organic baby spinach 312 g – \$6.99

Similar alternatives:
Regular baby spinach 312 g – \$4.99
Spinach and spring mix 312 g – \$4.49
Frozen chopped spinach 500 g – \$3.29

Allow users to choose:

- Exact match only
- Same brand
- Same type
- Similar alternative
- Cheapest equivalent

- Organic only
- Vegetarian or dietary restrictions

This can begin with category rules and later use product embeddings or an ML recommendation model.

5. “Buy Now or Wait?” indicator

Once you have several weeks of history, show:

Current price: \$5.99
Recent average: \$6.65
Lowest recorded: \$4.99
Typical sale cycle: Every 4 weeks

Recommendation: Good time to buy

Or:

Current price: \$6.99
Typical sale price: \$5.49
Last sale: 3 weeks ago

Recommendation: Consider waiting

Use careful language:

- Excellent price
- Good price
- Normal price
- Above usual price
- Limited history

Initially, this can be rule-based. Later, your ML model can predict the probability of a discount next week.

6. Personal Price Watchlist

Users save products they frequently buy:

Milk 4 L
Baby spinach
Paneer
Diet Coke
Rice
Yogurt

Then your app alerts them when:

- The price reaches their chosen limit
- The product hits its lowest price in 90 days
- A nearby store offers it cheaper

- A favourite item appears in a new flyer
- Several watchlist products are discounted at the same store

The last point is especially powerful:

“Five items on your list are on sale at Maxi this week. Estimated savings: \$14.20.”

That encourages users to return every week.

7. Best Store for Me

Instead of saying that one store is universally cheapest, create a personalized ranking based on what the user normally buys.

Example:

Based on your previous shopping lists:

1. Maxi – cheapest in 46% of your baskets
2. Walmart – cheapest in 32%
3. Super C – cheapest in 15%

You can also show category strengths:

Best for your vegetables: Adonis
Best for dairy: Maxi
Best for packaged food: Walmart
Best overall for your basket: Maxi

This becomes more intelligent as users create more lists.

8. Flexible product search

A user may search:

spinach

You should not mix everything into one confusing list.

Group results:

Fresh baby spinach
Fresh regular spinach
Organic spinach
Frozen spinach
Spinach mixes
Prepared spinach products

Then provide smart filters:

- Fresh or frozen
- Organic or regular
- Package size

- Brand
- Store
- Distance
- Unit price
- Sale only

Your search should understand:

```

baby spinach
spinach baby
fresh spinach
organic palak
palak leaves

```

This semantic search could become one of your ML features.

9. Household-aware recommendations

Ask users for only simple preferences:

```

Household size: 1, 2, 3-4, 5+
Shopping frequency: Weekly or biweekly
Preferred maximum stores: 1, 2, 3

```

Then recommend appropriate package sizes.

Example:

The 454 g spinach package has the lowest unit price, but the 142 g package may be better for one person because less food may be wasted.

This prevents the common mistake of treating the largest package as automatically best.

10. Expiry and food-waste intelligence

Flyers often promote bulk quantities. Buying more is not always saving.

Your app could estimate:

```

Large package savings: $1.20
Estimated unused amount: 35%
Effective saving after likely waste: $0.18

```

For early development, use category-based shelf-life estimates rather than ML.

Examples:

- Leafy greens: short shelf life
- Potatoes and onions: longer shelf life
- Frozen products: very long

- Pantry items: long

Present it as an estimate, not a guarantee.

11. Recipe-to-cart comparison

A user enters or selects a recipe:

Palak paneer for four people

Your app extracts:

Spinach
Paneer
Onion
Tomatoes
Cream
Spices

Then calculates:

- Cheapest store for the recipe
- Total recipe cost
- Cost per serving
- Missing pantry items
- Cheapest substitutions
- One-store versus multi-store options

Example:

Total cost: \$18.40
Estimated cost per serving: \$4.60

Best single store: Maxi
Cheapest mixed-store basket: Maxi + Adonis
Additional saving: \$3.10

This would be an excellent AI-powered feature and easy to demonstrate in a portfolio.

12. Budget-based shopping assistant

The user enters:

I have \$70 for groceries this week.

The app considers:

- Existing shopping list
- Preferred foods
- Dietary restrictions

- Current flyers
- Household size
- Store distance

Then it builds an affordable basket.

It could say:

Your original list: \$78.40

Suggested adjustments:

- Replace branded yogurt: save \$2.20
- Buy rice at Maxi: save \$3.10
- Choose 312 g spinach instead of organic: save \$2.00
- Replace one packaged snack: save \$2.50

New total: \$68.60

The system should suggest substitutions without pressuring users or making medical claims.

13. Price anomaly detector

Your crawler may occasionally capture errors:

Milk 4 L: \$0.59
Spinach 142 g: \$59.99

An anomaly system can flag:

- Impossible prices
- Unexpected package sizes
- Sudden 500% price changes
- Duplicate offers
- Unit parsing errors
- Mismatched products

This feature is mostly behind the scenes, but it makes the entire app more reliable.

Use rules first:

Flag price if:

- Change is above a category threshold
- Unit price is far outside normal category range
- Package quantity is missing
- Price differs drastically from historical median

Later, use anomaly-detection models.

14. Confidence score for every comparison

Your app should indicate how certain it is that two listings are identical.

Coca-Cola Zero 6 × 710 mL
Coke Zero Sugar 6-pack 710 mL

Match confidence: 98%

For weaker matches:

Possible similar product – verify package size
Match confidence: 74%

Only use high-confidence matches when highlighting the definite cheapest store.

This protects users from false comparisons such as:

- Different package sizes
- Organic versus non-organic
- Fresh versus frozen
- Brand versus store brand
- Different quantities

15. Weekly savings report

Give users a simple dashboard:

This week

Shopping-list total: \$64.20
Without comparison: \$76.80
Potential saving: \$12.60

Best store: Maxi
Best category deal: Produce
3 watched products reached good prices

Over time:

Estimated savings this month: \$48.30
Estimated savings this year: \$214.60

Use the user's actual selected basket rather than claiming savings solely because they viewed an offer.

Best feature combination for your app

I would develop the product in this order:

Version 1

Keep your existing exact-product comparison and add:

1. Unit-price normalization

2. Search grouping and filters
3. Product-match confidence
4. Price history
5. Good-price indicator

Version 2

Build:

1. Full shopping-list comparison
2. Best single-store basket
3. Maximum two-store basket
4. Distance-aware savings
5. Price alerts

Version 3

Add intelligent features:

1. Similar product substitutions
2. Buy-now-or-wait prediction
3. Recipe-to-cart assistant
4. Personalized store ranking
5. Budget-based basket generation

The standout idea

The most impressive experience would be:

The user uploads or types their whole grocery list. Your app identifies every item, finds equivalent products, compares package and unit prices, calculates the cheapest one-store and two-store baskets, considers travel distance, and explains exactly where the savings come from.

Example result:

Your list has 14 items.

Best one-store option:
Maxi – \$82.40

Cheapest overall:
Maxi + Adonis – \$75.10

Gross savings: \$7.30
Estimated extra travel cost: \$2.10

Real estimated savings: \$5.20

Recommendation:
Use Maxi only unless you are already travelling near Adonis.

That is far more useful than simply highlighting the lowest price and would make your application feel genuinely intelligent.

Ok got it, and any or all important buttons, banners, nav bar or any other widgets or anything other to add or must have or recommend to add on the page or dashboard of the website

Yes. Since you're building a **consumer-facing grocery platform**, the UI is just as important as the backend. One thing I've noticed is that many grocery comparison sites feel like spreadsheets. You can differentiate your app by making it feel more like a modern fintech dashboard (think Uber, Spotify, or Airbnb) while keeping it simple.

Homepage

Hero section

```
-----
Find the cheapest groceries near you.

Compare prices across 85+ Canadian stores
Save money every week.

[ Search Products ]    [ Create Shopping List ]

🔥 This week's best deals
💰 Average savings this week: $18.42
-----
```

Search Bar (Most Important)

Instead of a normal search bar:

```
🔍 Search products...

Recent Searches
• Milk
• Baby Spinach
• Diet Coke
• Eggs

Trending
🔥 Blueberries
🔥 Ground Beef
🔥 Butter
```

Autocomplete should suggest

- Product
- Brand
- Category

Example:

milk

Milk
2% Milk
Whole Milk
Almond Milk
Oat Milk

Filters (VERY important)

Users LOVE filters.

Store

- ☒ **Walmart**
- ☒ **Maxi**
- ☒ **Super C**
- ☒ **Adonis**

Price

- \$0-\$5**
- \$5-\$10**

Brand

- Dairyland**
- Neilson**
- Great Value**

Only on Sale

- ☒

Organic

- ☒

Distance

- 5 km**

Package Size

- 500g**
- 1kg**
- 2kg**

Product Card

Instead of only showing

Milk
\$5.99

show something like

 **Neilson 2% Milk**

★★★★☆

4L

Current Price

\$5.99

Lowest today

MAXI

Good Deal

Compared to 90 day average

Price History

View Details

Store Comparison Table

This is where users spend most of their time.

Store

Price

Distance

Savings

Maxi

\$5.99

3km

Cheapest

Walmart

\$6.49

5km

+\$0.50

IGA

\$6.99

1km

+\$1.00

Quick Action Buttons

On every product

 Save

 Price Alert

 Price History

 Add to Shopping List

Share

Price History Widget

Very clean

Lowest Ever
\$4.49
Highest
\$7.99
Average
\$6.22
Current
\$5.99

Then the graph underneath.

Deal Badge

Instead of every product looking the same

show badges

-  Great Deal
-  Fair Price
-  Expensive
-  Lowest in 90 Days
-  Store Exclusive

New

Limited Time

Weekly Deals Banner

 This Week's Biggest Savings
Blueberries ↓35%
Chicken Breast ↓28%
Butter ↓40%
See All Deals →

Shopping List Widget

Pinned on dashboard

My Shopping List

12 Items

Estimated Total

\$61.42

Cheapest Store

MAXI

Optimize List

Smart Banner

Random tips

 You could save \$9.45 this week by shopping at Maxi instead of Walmart.

Optimize

This increases engagement.

Nearby Stores Widget

Stores Near You

Maxi

2.3 km

IGA

1.4 km

Adonis

3.1 km

Category Cards

Instead of searching every time

 Dairy

 Meat

 Produce

 Bakery

 Drinks

 Pantry

 Snacks

Trending Searches

Trending

Blueberries

Eggs

Watermelon

Ground Beef

Chicken Breast

Personal Dashboard

After login

Welcome back Narendra

Money Saved

\$83

Products Tracked

15

Price Alerts

3

Shopping Lists

2

Favorite Store

Maxi

Price Alert Widget

Price Alerts

Milk ↓

Butter ↓

Eggs ↓

View All

Floating Search Button (Mobile)

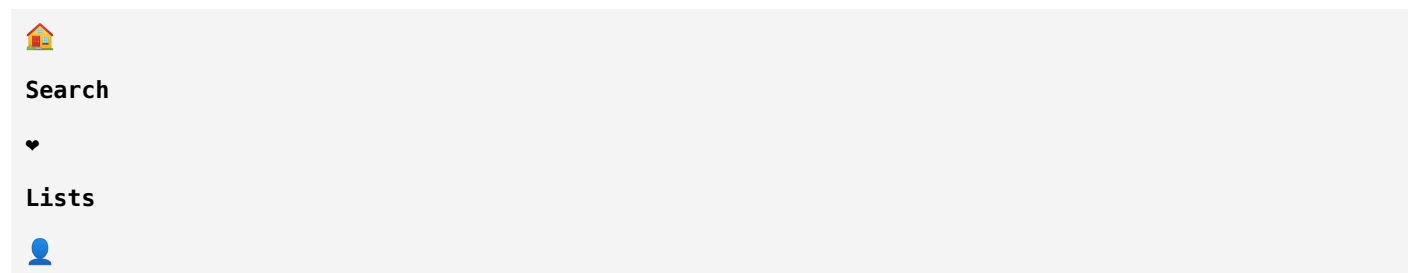
Very important.

Bottom

 Search Products

always visible.

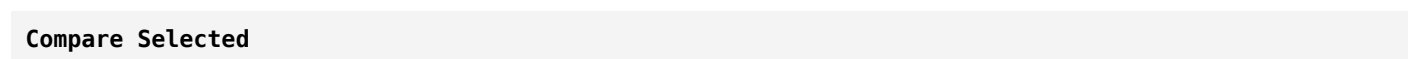
Bottom Navigation (Mobile)



Simple.

Compare Button

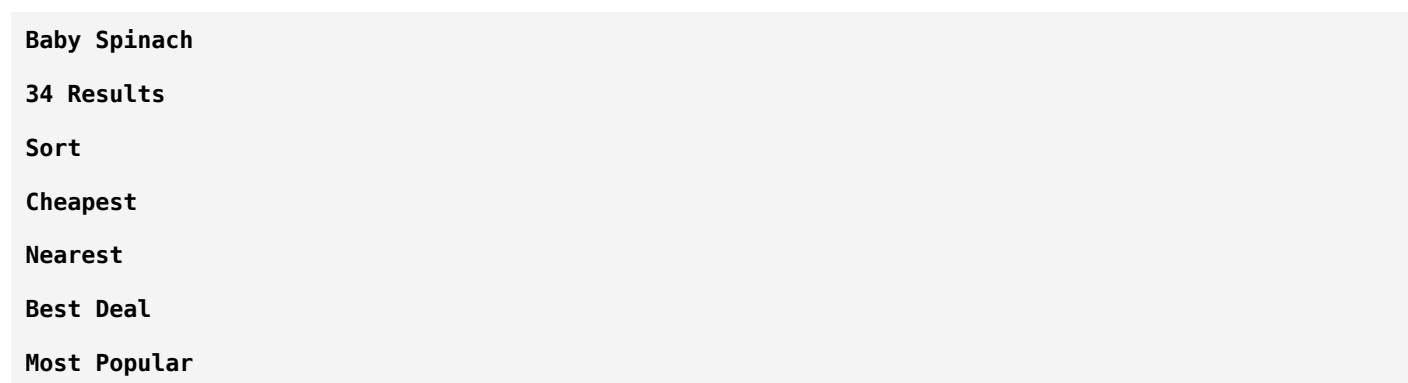
When users select products



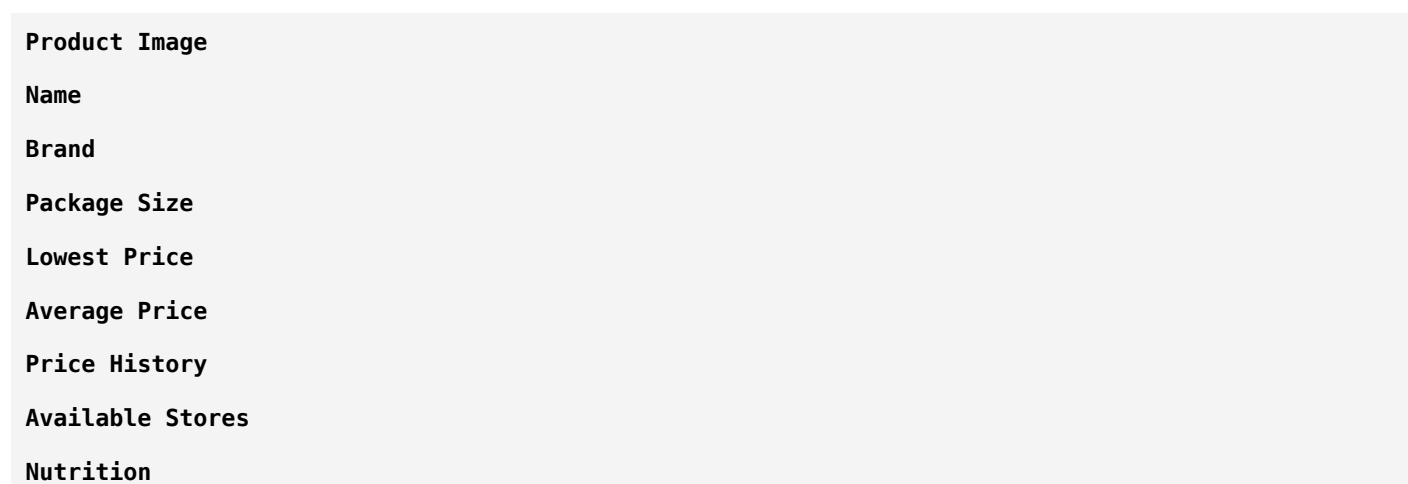
instead of forcing them to search individually.

Search Result Page

Top



Product Detail Page



Price Alerts

Similar Products

Footer

About

Privacy

Contact

FAQ

Supported Stores

API

Blog

Extra Dashboard Statistics

These make the app feel "alive."

Products Indexed

432,000

Stores

85

Weekly Flyer Items

112,000

Today's Price Updates

9,400

Average Savings

18%

Future AI Widget

One day

AI Shopping Assistant

Type

"I want groceries under \$80"

or

"I need ingredients for butter chicken"

Generate Basket

Gamification

People love stats.

This Month
You Saved
\$42.18
Compared to Average Shopper
Top 15%
Green Shopper Badge

★ If I were designing the homepage, I'd prioritize this layout

Navbar

Hero Search
Weekly Deals Banner
Categories
Trending Products
Recent Price Drops
Search Results
Shopping List Widget
AI Recommendation Banner
Nearby Stores
Footer

If I could add only one feature that would make your app feel premium, it would be this:

Savings Calculator Banner

Every time a user builds a shopping list, display a sticky banner at the top:

 You can save \$12.47 today
Best Store:
 Maxi
Estimated basket:
\$74.92
Optimize My Shopping →

It's simple, immediately valuable, and gives users a clear reason to keep using the app. It also reinforces the core purpose of your platform: helping people save money, not just compare prices.